WISEConv: Closing the Sparsity-to-Speedup Gap in Masked Convolution with Worklist-Driven Execution

Anonymous Author(s)

Abstract

Masked convolution promises to reduce inference latency by computing only positions an upstream policy marks active, yet existing paths sacrifice either throughput or useful-compute ratio, failing to translate sparsity into proportional latency reduction. We find that redirecting a dense tiled pipeline to compute only scattered active positions largely preserves per-tile throughput: irregularity is confined to only row-level indirection outside the reduction loop, leaving channel-contiguous access and weight-tile reuse identical to dense execution. Based on this finding, we present WISEConv, which drives the dense tiled pipeline with an active-output worklist to achieve both high useful-compute ratio and high throughput. To handle a construction cost that does not shrink with sparsity, WISEConv builds the worklist in a single mask-space pass and reuses it across compatible operators. To adapt tile configurations as the workload size shifts per frame on the device, WISEConv introduces on-device tile switching that trades off each candidate's gain against its selection cost. Across six GPUs and four workloads, WISEConv achieves a XX× geometric-mean latency reduction over the fastest baseline per workload-platform pair, reducing energy by XX–XX% on embedded GPUs.

Keywords: Sparse convolution, dynamic spatial sparsity, GPU runtime, deep learning systems, efficient inference

1 Introduction

Dense convolution dominates per-frame inference latency in many latency-critical vision tasks [3, 11, 41], yet on a given input much of that computation is spatially redundant. A common remedy is masked convolution: an upstream policy marks spatially active positions on the dense feature grid, and a dedicated sparse convolution operator updates only these positions in the dense output. The mask may derive from event-camera increments [9, 25], temporal differences [18, 31], or learned spatial gates [23, 38]. These mask sources promise large reductions in convolution work, yet existing masked-convolution systems [9, 23, 31, 38] rarely translate the sparsity into proportional latency reduction.

Dense GPU convolution obtains high *throughput* (operations per second) from coordinate-driven, tiled execution. Each tile enumerates a contiguous block of output coordinates; those coordinates fix each output position's receptive field and write location, while the reduction over channels and kernel offsets follows a regular datapath with coalesced loads and weight-tile reuse. In masked convolution, reducing latency proportionally requires sustaining this throughput while achieving a high *useful-compute ratio* (the fraction of issued operations that produce needed outputs), but the per-frame position selection imposed by the mask conflicts with the contiguous coordinate assumption that sustains throughput, placing these two axes in tension.

Existing systems sacrifice one axis for the other. *Gather-scatter* [23, 38, 42] extracts exactly the active positions into a packed representation, computes, and scatters the results back, but is bottlenecked by the memory traffic of materializing their receptive fields. *Tile skipping* [9, 18, 31, 33] suppresses tiles that are entirely inactive and retains the dense pipeline, but wastes computation on partially active tiles. On an RTX 3080 running YOLOv8m [20] at batch 4 under temporal difference masks [31], 82.2% of convolution work is unnecessary. Gather-scatter sustains only 43.7% of the dense pipeline's throughput, reducing latency by just 59.3%; tile skipping reaches higher throughput but wastes 32.6% of issued operations, reducing latency by 54.3%, slower than gather-scatter despite its throughput advantage (§2.2).

We find that redirecting a dense tiled pipeline to compute only scattered active positions achieves both a high useful-compute ratio and the throughput of dense execution. We collect these positions into a worklist and charaterize (§3) the resulting tiled execution on a desktop and an embedded GPU, identifying why: the irregularity is confined to one indirection per tile in the prologue, outside the reduction loop that drives coalesced channel loads and weight-tile reuse, so per-tile throughput matches dense execution. When the tile configuration follows the worklist length, convolution time is affine in the active ratio with a slope matching the dense marginal cost; the residual gap is an intercept of 9.0–25.8% of dense time per layer, dominated by launch latency and constant-cache misses, not by throughput loss.

While this finding establishes that worklist-driven tiled execution can sustain both axes, closing the gap end to end requires solving two further challenges. First, *worklist construction cost*: deriving the worklist from the mask requires scanning the whole grid and costs time independent of the active count, so construction's share of latency grows as activity falls and accumulates when every layer rebuilds. Second, *worklist consumption efficiency*: the affine property holds for a given number of active positions under a matching tile configuration, but in deployment this number is visible only on the device after construction and shifts across frames and layers. Reading it back to the host stalls the per-frame path; adapting on the device avoids the stall but incurs a non-negligible cost per decision point (§3).

Based on these findings, we present WISEConv (**W**orklist-dr**I**ven ma**S**k**E**d **Conv**olution), which drives the dense tiled pipeline with an active-output worklist to achieve both a high useful-compute ratio and high throughput by co-designing worklist construction and consumption. To reduce construction cost, a single mask-space pass fuses output-mask propagation with per-tile compaction, operating on only mask and coordinate metadata; compatible operators reuse the worklist instead of reconstructing it. To sustain consumption efficiency, a persistent compute grid consumes the worklist without host synchronization, and a cost-aware selection mechanism adapts the tile configuration per frame, adding a candidate to a layer only where its expected gain exceeds the selection overhead.

This paper makes the following contributions:

- We characterize worklist-driven tiled execution, identifying why computing scattered active positions preserves dense-level throughput and where the remaining gap comes from. These findings motivate and constrain the design (§3).
- We present WISEConv, which co-designs worklist construction, worklist-driven tiled compute, and cost-aware tile selection to jointly sustain high useful-compute ratio and high throughput (§5).
- We evaluate WISEConv on four workloads across six GPUs. WISEConv achieves a 2.5× (FP32) and 2.4× (FP16) geometric-mean speedup over the fastest baseline per pair, reducing energy by XX–XX% on embedded GPUs (§7).

2 Background and Motivation

2.1 Masked Convolution

Convolution runs inside the perception-action loop of many vision applications [3, 35, 39, 41, 45], where per-frame inference latency bounds system responsiveness [10–12, 19]. To cut this latency, masked convolution exploits dynamic spatial sparsity: an upstream policy emits a spatial mask, and the operator computes only the marked positions. The mask comes from sources including event-camera increments [9, 25], temporal residuals between video frames [4, 18, 30, 31, 40], or learned input-dependent gates [14, 23, 38, 42]. These sources differ in sensing modality and policy, but all expose the same dynamic spatial sparsity on a dense 2D feature grid, so a single masked-convolution runtime serves them all.

Masked convolution retains dense tensor layouts and arithmetic, restricting only which output coordinates are computed. It operates on an input feature tensor $\mathbf{x} \in \mathbb{R}^{H_{in} \times W_{in} \times C_{in}}$, a weight tensor $\mathbf{f} \in \mathbb{R}^{k \times k \times C_{in} \times C_{out}}$ of kernel size k, and an output $\mathbf{y} \in \mathbb{R}^{H_{out} \times W_{out} \times C_{out}}$ on a coordinate grid $\mathcal{G} = \{1, \ldots, H_{out} W_{out}\}$, where each coordinate $p \in \mathcal{G}$ indexes a spatial position (h, w) and has a $k \times k$ receptive field $\mathcal{N}(p)$ in $\mathbf{x}$. An upstream policy marks the coordinates to compute in an output mask $M_{out} \in \{0,1\}^{H_{out} \times W_{out}}$. Spatial gating predicts M_{out} directly; event and temporal policies instead supply an input mask $M_{in} \in \{0,1\}^{H_{in} \times W_{in}}$ whose active positions propagate their influence along the receptive field, so p stays active whenever any input in $\mathcal{N}(p)$ is active,

$$M_{out}[p] = \begin{cases} 1, & \exists r \in \mathcal{N}(p) \text{ s.t. } M_{in}[r] = 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

M_{out} thus identifies the output positions that the execution path is required to compute, each by aggregating $\mathbf{x}$ over $\mathcal{N}(p)$ with the weights $\mathbf{f}$.

An execution path issues computation over a sequence of position slots C, where each slot is drawn from $\mathcal{G} \cup \{\perp\}$; $\perp$ denotes a padding slot that carries no output coordinate. The path must satisfy the *coverage contract*: every active coordinate is included, $\{p : M_{out}[p] = 1\} \subseteq \{c \in C : c \neq \perp\}$. For each non-$\perp$ slot $p \in C$ it computes

$$\mathbf{y}[p] = \sum_{i=1}^{k^2} \mathbf{x}[\mathcal{N}_i(p)]\, \mathbf{f}_i, \tag{2}$$

where $\mathcal{N}_i(p)$ is the i-th position in $\mathcal{N}(p)$ and $\mathbf{f}_i \in \mathbb{R}^{C_{in} \times C_{out}}$ the corresponding weight slice. How unrequested positions are treated is policy-specific: event and temporal execution retains prior state, while learned gating merges results into a residual. Coverage is therefore the operator-level correctness condition; the policy determines the treatment of all other positions. Beyond coverage, a path may issue inactive or $\perp$ slots; dense convolution is the extreme where $\{c \in C : c \neq \perp\} = \mathcal{G}$. The policy fixes the accuracy and required work.

2.2 The Sparsity-to-Latency Gap

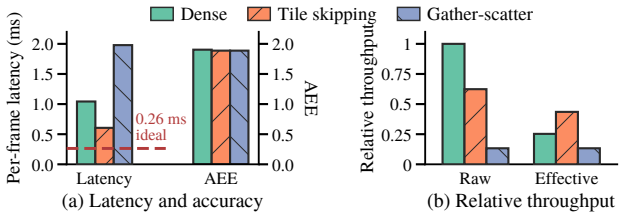


Figure 1. TODO: YOLOv8m [20] at batch 4 using the DeltaCNN [31] temporal difference mask policy on WildTrack 640 and RTX 3080, comparing cuDNN (Dense), DeltaCNN tile skipping, and MMCV gather-scatter [6]. (a) Mean full-model per-frame latency. The dashed line marks dense latency scaled by the active ratio aggregated across layers; the sparse paths use the same masks. (b) Raw throughput T and effective throughput T_{eff}, each normalized to dense raw throughput. The dense useful-compute ratio equals the active ratio set by the mask policy.

Existing systems execute a mask along one of two paths. *Tile skipping* [9, 18, 24, 30, 33, 40] (Figure 7b) inherits the dense pipeline's tile-based computation without additional

metadata passes, skipping a tile only when all its coordinates are inactive and otherwise computing the whole tile; DeltaCNN [31] fuses a selective path into the kernel, taken when a tile is estimated to be mostly empty, to cut wasted work. *Gather-scatter* [23, 42] (Figure 7c) instead abandons the dense pipeline: it extracts exactly the active coordinates into a packed representation, runs a separate GEMM, and scatters the results back, removing the inactive arithmetic but adding extraction, irregular access, and writeback overhead; DynConv [38] adopts a model structure suited to gather-scatter, where one gather feeds several consecutive layers before a single scatter, amortizing that overhead.

A policy fixes the active ratio α, defined as the fraction of output-grid positions marked active, whereas a concrete execution path determines the useful-compute ratio η, defined as the fraction of issued slots that correspond to required outputs:

$$\alpha = \frac{\|M_{out}\|_0}{|\mathcal{G}|}, \quad \eta = \frac{\|M_{out}\|_0}{|C|}. \tag{3}$$

For $|C| > 0$, an ideal path achieves $\eta = 1$ by issuing only required outputs; dense convolution, which issues the entire grid, has $\eta \approx \alpha$. An execution path also determines its throughput T, which combines with η to yield the effective throughput T_{eff}:

$$T = \frac{2|C|k^2 C_{in} C_{out}}{t}, \quad T_{\text{eff}} = \eta T. \tag{4}$$

where t denotes the latency of the execution path, including auxiliary mask and coordinate processing, k is the filter kernel size, and C_{in} and C_{out} are the input and output channel counts respectively. The factor of two counts each multiply-accumulate as two FLOPs. Consequently, T_{eff} directly characterizes the processing rate of required outputs and therefore determines the realized latency.

Figure 1 makes this concrete on RTX 3080 for YOLOv8m [20] at batch 4 under temporal difference masks [31], where 82.2% of convolution work is unnecessary. Tile skipping holds throughput at XX% of dense but at a useful-compute ratio of only 67.4%; gather-scatter lifts the useful-compute ratio to one, yet extraction traffic collapses its throughput to 43.7% of dense. Neither path efficiently converts the GPU's capability into effective throughput.

2.3 Distinction from Related Sparse Execution

WISEConv differs from other sparse-execution work in representation and sparsity type. 3D sparse-convolution systems [8, 17, 34, 36, 37, 43] operate on sparse point-cloud coordinates; sparse matrix multiplication [13, 15, 32] targets structural sparsity in graphs or pruned weights; structured weight pruning [27] exploits fixed patterns; sparse-format compilers [22, 44] encode sparsity in tensor formats. DPACS [16] targets the same dynamic 2D spatial sparsity but designs a custom dataflow accelerator to handle the resulting irregular access patterns. Our characterization (§3.2) shows that on commodity GPUs the irregularity is confined to row-level indirection outside the reduction loop, so a worklist-driven execution model achieves dense-level throughput without hardware changes.

Fig. char-affine: 4 subplots (FP32/FP16 × 3080/AGX), YOLOv8m b1. Each: t/t_{dense} vs. α (layer-averaged), lines: proportional diagonal, per-length best + affine fit, dense-tuned config.

Figure 2. Convolution latency averaged across layers, normalized to dense, on YOLOv8m at batch 1. Each panel shows the per-length best (with affine fit), the dense-tuned configuration, and the best single configuration under a uniform worklist-length assumption.

3 Characterizing Worklist-Driven Convolution

Before presenting WISEConv, we characterize what worklist-driven tiled convolution achieves and where it falls short. We study the compute stage (a dense tiled implicit-GEMM kernel redirected to compute only positions from a worklist; §5.3) and the construction stage (a mask-to-worklist pass) separately, so that each finding belongs to the execution model, not to a particular implementation. FP16 uses `mma.sync` tensor cores; FP32 is a SIMT kernel of the same structure. Each configuration is timed in isolation inside CUDA graphs on RTX 3080 (68 SMs) and Jetson AGX Orin (16 SMs) at locked clocks, using YOLOv8m [20] at batch 1 on WildTrack [5] unless noted otherwise. Dense reference: the faster of the same kernel on the full grid and cuDNN non-Winograd.

3.1 Affine Latency Requires Following the Length

A worklist kernel issues only active outputs ($\eta \approx 1$), so proportional speedup means keeping dense throughput. When we grant the kernel its best configuration at every worklist length, its latency averaged across layers is affine in α, well fitted by $t/t_{\text{dense}} = a\alpha + b$. On RTX 3080, $a = 0.794$, $b = 0.258$ (FP32) and $a = 0.643$, $b = 0.417$ (FP16); on AGX Orin, $a = 0.917$, $b = 0.085$ (FP32) and $a = 0.830$, $b = 0.267$ (FP16), with per-layer R^2 medians of 0.97–0.995 (Figure 2). The slope a matches the dense marginal cost; the entire departure from proportional is the intercept b, a per-layer fixed cost analyzed in §3.4.

This affine behavior holds only when the configuration follows the worklist length. The dense-tuned configuration runs 2.3× (FP32) and 1.6× (FP16) slower than the per-length best at low activity on RTX 3080, and 4.0× and 1.9× on AGX

Fig. char-reduction: representative layers, RTX 3080 FP32. (a) sectors/request: worklist vs. dense. (b) DRAM bytes and L2 traffic: worklist vs. dense.

Figure 3. Per-tile memory behavior on representative layers (RTX 3080 FP32). (a) Sectors per request match dense. (b) DRAM traffic does not increase significantly; L2 absorbs the overlap of scattered receptive fields.

Orin (Figure 2): large tiles amortize weight loads over many rows, but when rows are few, smaller tiles and split-K shorten the chain each SM executes. The best single configuration under a uniform worklist-length assumption narrows the gap but remains XX–XX% slower than the per-length oracle.

Insight 1. *The latency of worklist-driven tiled convolution is affine in the active ratio at dense marginal cost only when the tile configuration follows the worklist length.*

3.2 Why Per-Tile Throughput Matches Dense

The worklist changes only the base address of each row; under a channels-last layout, the reduction loop over k^2 taps and channel slices keeps the dense kernel's addressing, so channel coalescing and weight-tile reuse are untouched. Counters confirm it: a worklist tile executes the same instructions as a dense tile (within 0.5%), with sectors per request matching dense (Figure 3a). DRAM traffic does not increase significantly over dense: neighboring active positions share overlapping receptive fields, and L2 absorbs the redundant accesses. L2 traffic is higher than dense because the worklist's spatial scattering reduces tile-local reuse, but this does not surface at DRAM (Figure 3b).

Insight 2. *Irregularity is confined to row-level indirection outside the reduction loop, leaving channel-contiguous access and weight-tile reuse identical to dense execution.*

3.3 Order Is a Second-Order Effect

Scattered worklist entries could hurt L1 reuse by placing positions with non-overlapping receptive fields into the same tile. We test this by comparing three orderings of the same worklist: tile-local (dense traversal order within each spatial tile, arbitrary order across tiles), globally sorted in dense traversal order, and randomly shuffled. Across 3,560 layer-frame samples on RTX 3080, global sorting runs at 0.999× the tile-local order (median) and shuffling at 1.004–1.007×; model-level totals are within 2.2%. All three move identical DRAM bytes; the difference is only in L1 hit rate. On AGX Orin, shuffling slows wide early layers by up to 1.33× because

Fig. char-order: latency relative to construction order for sorted / shuffled / uniform worklists; inset: DRAM, L1, L2 on representative layers.

Figure 4. Latency of four worklists with the same length and tile on real frames. Order changes L1 hits, never DRAM bytes.

Fig. char-bound: (a) intercept breakdown (launch + single-tile chain, stall reasons). (b) Left axis: construction time / oracle conv time vs. α; right axis: absolute construction time vs. α (flat).

Figure 5. (a) Composition of the intercept: an empty kernel spends 92% of stall cycles on constant-cache misses; a single tile adds barriers and memory fences. (b) Construction cost is nearly constant in absolute time (right axis), so its ratio to the oracle convolution time grows as activity falls (left axis).

Table 1. Intercept composition: latency and stall breakdown at $n_W = 0$, one tile, and $\alpha = 1$ (Nsight Compute, RTX 3080 and AGX Orin).

TODO: table from logs/wise2_intercept/

the extra L1-to-L2 traffic is exposed by the lower L2 bandwidth, but the tile-local order already recovers everything a global order would (0.993–0.999×).

Insight 3. *Worklist order has negligible impact on DRAM traffic; a tile-local order recovers all of a global order's benefit.*

3.4 The Residual Gap Is a Per-Layer Fixed Cost

The intercept b consists of launch latency and a single-tile dependent chain. An empty kernel spends 92% of its stall cycles on constant-cache misses (the parameters are cold); a single tile adds barriers and, with split-K, memory fences, with long-scoreboard stalls at only 6–27% (Table 1). The intercept is 10–17% of a median YOLOv8m layer's dense time on RTX 3080, smaller on AGX Orin where each layer is larger relative to the device.

Fig. char-cost: CDF of per-layer gain from per-length selection over best single config (μs); vertical lines: per-layer cost of host readback, updatable node, conditional node.

Figure 6. Per-layer gain of selecting the configuration per worklist length over the best single configuration (CDF over layers, RTX 3080), against the per-layer cost of each selection mechanism.

Construction adds a second fixed cost. Each mask-to-worklist pass takes 1.7–3.2 μs on RTX 3080 and 4.2–23.6 μs on AGX Orin, nearly independent of activity (Figure 5b, right axis). As the oracle convolution time shrinks with α, construction's share rises (Figure 5b, left axis): from XX% at $\alpha = 0.5$ to XX% at $\alpha = 0.1$ on RTX 3080 FP32.

Insight 4. *The residual gap comes from per-layer fixed costs independent of activity: launch latency in the compute intercept, and near-constant construction time.*

3.5 Following the Length Is Not Free

Insight 1 requires the configuration to follow the worklist length, but the length exists only on the device and changes at every layer of every frame. Every mechanism that selects a kernel from it pays per decision point: reading the length back to the host stalls the per-frame path; a device-updatable kernel node costs 1.9 μs per layer on RTX 3080; a conditional graph node costs 9.5 μs (RTX 3080) and 17–19 μs (AGX Orin). Fusing candidates into one kernel avoids the per-layer cost but allocates registers and shared memory for the largest path, slowing every tile by 5–6%. These costs are comparable to the median per-layer gain from per-length selection (Figure 6), so the number of candidates and selection points a frame can afford is bounded by the mechanism, not by the oracle.

Insight 5. *Every mechanism for following the worklist length incurs a per-decision-point cost often comparable to the per-layer gain from adaptation.*

Insights 4 and 5 together mean that current hardware cannot reach the oracle affine line without trading off selection cost against adaptation gain. Two architectural capabilities would close this gap: reducing the per-layer launch and synchronization floor that dominates the intercept (Insight 4), and providing a lightweight launch predicate evaluated before CTA resource allocation so that following the worklist length no longer competes with the gain it targets (Insight 5).

4 WISEConv Overview

Dense tiled convolution already consumes a stream of output coordinates: each coordinate determines the receptive-field reads and output write, while its reduction over channels and kernel offsets remains regular. WISEConv exploits this interface by replacing the dense coordinate sequence with a worklist of requested output positions, thereby introducing position-level selectivity into the tiled pipeline. Removing unrequested positions from the schedule raises the useful-compute ratio, but issuing fewer operations alone does not guarantee lower latency. To translate selectivity into latency reduction, the worklist must be inexpensive to construct and must be consumed with both a high useful-compute ratio and high throughput. WISEConv therefore co-designs a construction stage and a compute stage around these interdependent requirements, as Figure 7 summarizes.

Construction stage. To prevent worklist construction from erasing the savings of selective compute, WISEConv confines construction to mask and coordinate metadata (Figure 7d, ①), never reading feature or weight tensors. In one mask-space pass, it fuses the output-mask propagation in Eq. 1 with tile-local compaction, producing M_{out} and a worklist $\mathcal{W}$ (the figure shows the compaction step; mask propagation precedes it but is omitted); the requested coordinates from each spatial tile form a segment in local dense-traversal order, preserving spatial grouping without global ordering. WISEConv writes $\mathcal{W}$ into a preallocated buffer large enough for the full output grid. A device-resident counter records $n_{\mathcal{W}} = |\mathcal{W}|$, so construction needs neither runtime allocation nor a host-visible active count.

Because construction cost scales with the spatial grid rather than the active count or channel widths, its share of latency grows as activity falls and accumulates when every layer rebuilds. WISEConv therefore propagates and reuses the resulting mask and worklist metadata across compatible operators instead of reconstructing it, amortizing one construction across the compatible operator sequence. The fused pass lowers the cost of each construction, while cross-operator reuse reduces how often it occurs; together, they address the Worklist Construction Cost challenge.

Compute stage. The compute stage (Figure 7d, ②) replaces only the coordinate source of a dense tiled implicit-GEMM pipeline with the worklist; tensor layouts, weight access, and the per-position channel-and-kernel reduction remain unchanged from the dense path.

The worklist length varies per frame with the input mask: the tile count is unknown before launch and device-resident after construction. Synchronizing it to the host for decomposition selection would insert a per-operator stall on the critical path, yet a fixed decomposition loses efficiency across the resulting range of tile counts. WISEConv builds on Stream-K [28], which addresses GPU underutilization when tiles

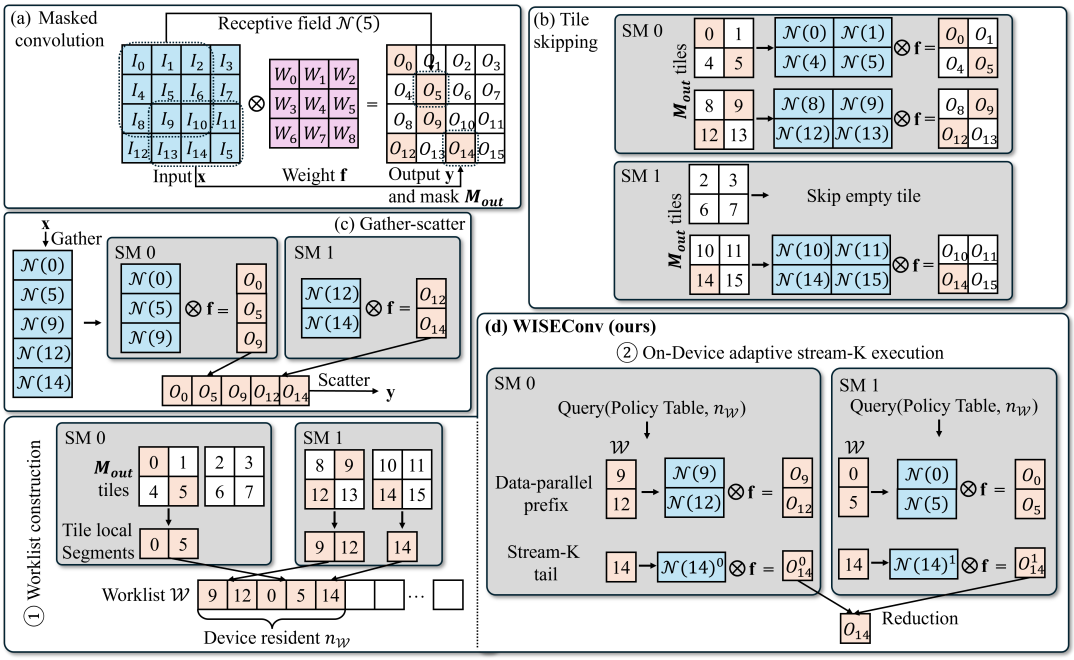


Figure 7. Overview of WISEConv and baseline execution paths on a two-streaming-multiprocessor (SM) example. (a) Masked convolution: M_{out} marks active output positions (colored); the operator computes only these from $\mathbf{x}$ and $\mathbf{f}$. (b) Tile skipping skips only fully inactive tiles; a single active position triggers computation for that tile. (c) Gather-scatter packs exactly the receptive fields of active positions, computes, and scatters results back. (d) WISEConv: ① constructs a tile-local worklist and ② consumes it via on-device adaptive Stream-K execution, where data-parallel tiles complete a full reduction each while Stream-K tail tiles split their reduction across SMs (O^0_{14}, O^1_{14}) and merge the partials.

cannot fill all SMs but assumes the tile count is host-visible at launch. WISEConv designs *on-device adaptive Stream-K execution*: it launches a persistent grid sized to SM residency, and the kernel reads the device-resident worklist length, looks up an offline-calibrated policy table, and selects its decomposition per frame without host involvement. This sustains GPU throughput as the worklist length varies from frame to frame.

Neither stage suffices alone: excessive construction cost, a low useful-compute ratio, or low consumption throughput can each erase the benefit of issuing less convolution work. By controlling all three jointly, WISEConv converts upstream spatial sparsity into end-to-end latency reduction.

5 WISEConv Design

The design follows the worklist from its construction (§5.1) and propagation across compatible operators (§5.2) to its consumption by the convolution pipeline (§5.3).

5.1 Tile-Local Worklist Construction

Constructing a tight schedule already requires a scan of the output grid, but the resulting coordinates must also retain enough spatial structure for efficient consumption. Globally stable compaction preserves the complete dense traversal order but requires a prefix scan or another pass to obtain device-wide offsets; per-coordinate appends avoid that coordination but scatter neighboring positions. Materializing active receptive fields, as gather-scatter does, regularizes the compute input at the cost of feature traffic proportional to the active count and $k^2 C_{in}$. The useful middle ground is therefore a metadata-only schedule with local rather than global ordering.

WISEConv realizes this with contiguous, tile-local worklist segments. Because the number of active positions is not known before the pass, it preallocates a coordinate buffer with capacity $|\mathcal{G}| = H_{out} W_{out}$, the static upper bound on $|\mathcal{W}|$. Algorithm 1 details the resulting single pass. Line 1 initializes the device-resident length counter. Lines 2–7 partition the output grid into $B_\tau \times B_\tau$ construction tiles, derive the corresponding entries of M_{out} using Eq. 1, and compact each tile's active coordinates into a local sequence $\mathcal{P}_\tau$ in dense-traversal order. When a learned gate supplies M_{out} directly [23, 38], Lines 4–5 are omitted and Line 7 compacts the supplied mask. Lines 8–11 count the compacted coordinates, atomically reserve $[\text{base}, \text{base} + n_\tau)$ from the counter, and copy $\mathcal{P}_\tau$ into that interval. After all tiles complete, the reserved intervals collectively store $\mathcal{W}$, and the counter

Algorithm 1 Tile-Local Worklist Construction

Require: Input mask M_{in}, output grid $\mathcal{G}$, tile size B_τ
Ensure: Mask M_{out}, worklist $\mathcal{W}$, length $n_{\mathcal{W}} = |\mathcal{W}|$

$n_{\mathcal{W}} \leftarrow 0$
for each $B_\tau \times B_\tau$ spatial tile τ **in parallel do**
  for each position $p \in \tau$ **in parallel do**
    active$(p) \leftarrow \exists\, r \in \mathcal{N}(p) : M_{in}[r] = 1$
    $M_{out}[p] \leftarrow$ active(p)
  end for
  $\mathcal{P}_\tau \leftarrow$ Compact$(\tau,\, M_{out})$
  $n_\tau \leftarrow |\mathcal{P}_\tau|$
  base $\leftarrow$ AtomicAdd$(n_{\mathcal{W}}, n_\tau)$
  for each $0 \le j < n_\tau$ **in parallel do**
    $\mathcal{W}[\text{base} + j] \leftarrow \mathcal{P}_\tau[j]$
  end for
end for

records $n_{\mathcal{W}}$. Because the layer shape fixes the buffer capacity and launch geometry while only the device observes the data-dependent length, construction requires no runtime allocation or host-visible active count.

To relate the worklist to the useful-compute ratio of Eq. 3, we insert its length $n_{\mathcal{W}}$ between the required-position count and the issued-slot count. For $n_{\mathcal{W}} > 0$,

$$\eta = \underbrace{\frac{\|M_{out}\|_0}{n_{\mathcal{W}}}}_{\eta_{\text{construct}}} \cdot \underbrace{\frac{n_{\mathcal{W}}}{|C|}}_{\eta_{\text{compute}}}. \tag{5}$$

We analyze $\eta_{\text{construct}}$ (required positions vs. worklist entries) here and η_{compute} (worklist entries vs. issued slots) in §5.3; the factorization decomposes η only, with all stage costs reflected in throughput T.

Because construction tiles partition the output grid and use disjoint reservations, each active coordinate appears exactly once, so $n_{\mathcal{W}} = \|M_{out}\|_0$. Fresh construction therefore has $\eta_{\text{construct}} = 1$ for a nonempty worklist.

WISEConv thus confines its ordering guarantee to each contiguous segment: coordinates retain the tile's dense traversal, while independently reserved segments provide no cross-boundary locality guarantee. A compute tile of B_M positions (the output-position tile width used by the compute stage, §5.3) may therefore span coordinates from multiple construction segments when segments are shorter than B_M; the preserved locality is intra-segment contiguity, not compute-tile-to-construction-tile alignment. As shown in §7.4, this tile-local order achieves compute-stage throughput comparable to global ordering, making the additional ordering pass an unfavorable end-to-end trade-off.

5.2 Cross-Operator Worklist Reuse

Each construction pass examines all $H_{out}W_{out}$ positions and, when deriving M_{out} from M_{in}, performs $O(k^2)$ mask work per position. This channel-independent grid scan contrasts with required convolution work, which scales with the active ratio, spatial extent, and both channel dimensions. WISEConv therefore amortizes construction by reusing one worklist across each compatible layer sequence.

For layer l, let $\mathcal{G}^l$ denote its output grid, M^l_{out} its output mask, $\mathcal{W}^l$ its worklist, and $n^l_{\mathcal{W}}$ its device-resident length. WISEConv propagates the mask and worklist metadata alongside the feature tensor. Whether two consecutive layers are reuse-compatible depends on their operator types, not on runtime mask values. An operator that preserves the output grid and does not expand the active set guarantees, for any input mask, that $\mathcal{G}^{l+1} = \mathcal{G}^l$ and

$$\{p \in \mathcal{G}^{l+1} \mid M^{l+1}_{out}[p] = 1\} \subseteq \{p \in \mathcal{G}^l \mid M^l_{out}[p] = 1\}. \tag{6}$$

WISEConv checks this condition once at model-conversion time. Because $\mathcal{W}^l$ covers M^l_{out}, Eq. 6 guarantees coverage for M^{l+1}_{out}. WISEConv therefore forwards $\mathcal{W}^{l+1} = \mathcal{W}^l$ and $n^{l+1}_{\mathcal{W}} = n^l_{\mathcal{W}}$ without reconstruction and applies the condition recursively at each subsequent layer.

Equality in Eq. 6 covers same-grid 1×1 convolutions, mask-preserving activations, and inference-time normalization, leaving $\eta_{\text{construct}}$ unchanged. Strict inclusion can result from mask-narrowing operators in the upstream policy, such as the activation-fused update truncation that DeltaCNN [31] uses to increase downstream sparsity. The inherited worklist remains a valid superset, trading a lower $\eta_{\text{construct}}$ for one fewer construction pass without violating coverage. Because compute gates all output writes by the exact mask, positions scheduled but no longer active under the current mask participate in reduction yet produce no store; the superset schedule therefore cannot alter any inactive position's observable state.

A change in the output grid violates the geometry constraint, while receptive-field expansion may violate Eq. 6 by introducing positions outside the upstream mask; either case invokes the construction pass of §5.1 to emit a new tight worklist. A lower $\eta_{\text{construct}}$ alone does not trigger reconstruction because tightening a valid superset would pay another grid scan. Common receptive-field-expanding layers such as 3×3 convolutions still create rebuild points throughout the network, leaving device-resident worklist lengths variable across layers and frames. Compute must therefore handle a dynamic problem size.

5.3 On-Device Adaptive Stream-K Execution

Construction and reuse fix $\eta_{\text{construct}}$; the compute stage controls η_{compute} and throughput T. Dense convolution maps to an implicit GEMM with M output positions, $N = C_{out}$ output channels, and reduction depth $K = k^2 C_{in}$. An output tile covers B_M positions and B_N output channels; the reduction over K proceeds in steps of B_K. In the standard *data-parallel* decomposition, each of the $\lceil M/B_M \rceil \times \lceil N/B_N \rceil$ output tiles is assigned to one thread block, which performs the full K reduction; split-K instead replicates each tile across S_k blocks

that each reduce a disjoint K-slice. Because M, N, and K are static, the tile shape (B_M, B_N), reduction step B_K, and split factor can be selected once offline by profiling and keeping the fastest.

In worklist convolution, N and K remain static, but the M dimension becomes $n_{\mathcal{W}}$, the device-resident worklist length written by construction, which varies by an order of magnitude or more across frames of a single layer. The offline assumption breaks: a single configuration cannot serve the full range. Two failure regimes bracket the problem. When activity is high and output tiles are plentiful, K-axis parallelism adds coordination and accumulation overhead without improving occupancy. When activity is low and tiles are scarce, data-parallel execution leaves streaming multiprocessors (SMs) without work because the tile count no longer fills a full *wave* (the set of thread blocks that co-reside across all SMs), referred to as the problem of wave quantization. Adapting between these regimes requires knowing $n_{\mathcal{W}}$; transferring it to the host inserts a per-operator synchronization on the critical path.

WISEConv builds on Stream-K [28], the established solution to wave quantization in dense GEMM. Stream-K launches a *persistent grid* of g_{res} blocks determined by the device's SM occupancy (not the tile count) and partitions all tiles' K-iterations into S_k units consumed by grid stride; partial K-slices fill SMs that would otherwise sit idle after the data-parallel waves complete. Its hybrid variant [1] restricts the K-splitting to the partial tail wave, handling complete waves data-parallel to avoid partial-reduction overhead where it is unnecessary. In worklist convolution the tile count changes every frame, so the wave boundary shifts with it. This shifting boundary is the operating point of WISEConv's *on-device adaptive Stream-K execution*: the compute kernel reads the exact worklist length on the device, determines the current wave boundary, and selects between data-parallel, Stream-K, and hybrid Stream-K decomposition accordingly, without host involvement.

WISEConv fixes the tile configuration $\hat{q}^l = (B_M, B_N, B_K)$ per layer at deployment and adapts only the decomposition mode on-device. What changes per frame is how the K reduction across output tiles is assigned to thread blocks, selected from three modes:

- *Data-parallel*: each block owns a complete output tile and reduces the full K range.
- *Stream-K*: each output tile's K-iterations are partitioned into S_k units; the persistent grid consumes all units by grid stride, and partial results from blocks sharing a tile are resolved through the workspace.
- *Hybrid Stream-K*: a data-parallel prefix handles complete waves; a streaming tail distributes the remaining tiles' K-iterations across one wave of blocks, eliminating partial-wave waste without applying the streaming overhead to the entire grid.

Both $\hat{q}^l$ and the per-length mode choice are calibrated offline. WISEConv selects $\hat{q}^l$ from a small catalog $\mathcal{Q}$ by profiling each configuration at the full worklist $\mathcal{W}^l_{\text{full}}$ (all $|\mathcal{G}^l|$ output positions) and keeping the fastest. Let $t(q, \mathcal{W})$ denote the minimum kernel latency of configuration q on worklist $\mathcal{W}$ over all mode candidates:

$$\hat{q}^l = \arg\min_{q \in \mathcal{Q}} \; t(q,\ \mathcal{W}^l_{\text{full}}). \tag{7}$$

At the full worklist $\eta_{\text{compute}} \approx 1$, so this step essentially maximizes throughput T, ensuring competitiveness with dense execution. The choice of B_M also determines η_{compute} at every other worklist length: the issued slot count is $|C| = B_M \lceil n^l_{\mathcal{W}} / B_M \rceil$, so $\eta_{\text{compute}} = n^l_{\mathcal{W}} / |C|$ can drop as $n^l_{\mathcal{W}}$ shrinks relative to B_M. Fixing B_M per layer is a necessary constraint, but per-shape profiling partially mitigates the cost: a large B_M on a small output grid often produces too few tiles to fill the device even at full worklist, so such layers naturally select a smaller B_M that also limits η_{compute} loss at low activity. The mode table below compensates for the remaining throughput gap by adapting the K-splitting strategy on-device.

Within $\hat{q}^l$, WISEConv builds a *policy table* P^l indexed by tile count $s = \lceil n^l_{\mathcal{W}} / B_M \rceil \times \lceil C_{out} / B_N \rceil$. Each entry stores a tail-tile count s_{tail} and a split factor S_k: $s_{\text{tail}} = 0$ yields data-parallel execution, $s_{\text{tail}} = s$ yields full Stream-K, and $0 < s_{\text{tail}} < s$ yields hybrid Stream-K with $s - s_{\text{tail}}$ data-parallel prefix tiles. WISEConv profiles a set of sampled tile counts and interpolates to fill the remaining entries:

$$P^l[s] = \arg\min_{s_{\text{tail}},\, S_k} \; t(\hat{q}^l,\ s,\ s_{\text{tail}},\ S_k), \tag{8}$$

where the minimization ranges over candidate tail-tile counts and split factors. All candidates process the same output tiles and differ only in how K-iterations are assigned to blocks, so η_{compute} is invariant and minimizing latency directly maximizes $\eta_{\text{compute}}\, T$.

Algorithm 2 presents the hybrid mode, which subsumes the other two: $s_{\text{tail}} = 0$ yields data-parallel (empty tail), while $s_{\text{tail}} = s$ yields full Stream-K (no data-parallel prefix); the policy table selects among the three based on the current tile count. Algorithm 2 omits boundary-tile padding when $n_{\mathcal{W}}$, C_{in}, or C_{out} is not divisible by B_M, B_K, or B_N, respectively; the implementation masks out-of-bounds accesses in these tiles.

Lines 1–4 of Algorithm 2 read $n_{\mathcal{W}}$, derive the tile count, and look up (s_{tail}, S_k) from P^l. The persistent grid first processes complete waves data-parallel (Lines 7–18): each block decodes its position and channel tile indices, reads B_M output coordinates from the worklist $\mathcal{W}$, and reduces the full reduction depth, writing the result directly to the output. The remaining partial-wave tiles enter the Stream-K tail (Lines 21–36): the reduction depth of each tile is partitioned among S_k contributors (the split factor selected by the policy table), and each block reduces its assigned slice into a pre-allocated workspace slot for that tile. Each slot holds

a $B_M \times B_N$ partial accumulator in accumulation precision (typically FP32); the workspace $\mathbf{Z}$ is sized to $\max(S_k) \times |\mathbf{y}^l|$ elements per layer, where $\max(S_k)$ is the largest split factor in the policy table P^l. A per-tile completion counter tracks how many contributors have finished; the last block to arrive loads all partial fragments, reduces them, applies the output epilogue, and writes the final result (Lines 33–37). Both write paths guard stores with M_{out} (§5.2). No device-wide barrier is needed; data-parallel tiles bypass the workspace entirely.

Algorithm 2 On-Device Adaptive Hybrid Stream-K

Require: $\mathcal{W}$, M_{out}, P, g_{res}, B_M, B_N, B_K, $\mathbf{x}$, $\mathbf{f}$, k, $\mathbf{Z}$
Ensure: $\mathbf{y}$

1: $n_{\mathcal{W}} \leftarrow$ **read** device-resident worklist length
2: $s_M \leftarrow \lceil n_{\mathcal{W}}/B_M \rceil$; $s_N \leftarrow \lceil C_{out}/B_N \rceil$; $s \leftarrow s_M \times s_N$
3: $(s_{\text{tail}}, S_k) \leftarrow P[s]$ ▷ *policy lookup*
4: $K_{\text{iters}} \leftarrow k^2 \lceil C_{in}/B_K \rceil$
5: ▷ ——————————*Data-parallel prefix*——————————
6: $s_{\text{dp}} \leftarrow s - s_{\text{tail}}$
7: **for** $b = \text{bid},\ \text{bid}+g_{\text{res}},\ \ldots < s_{\text{dp}}$ **do**
8: $(b_m, b_n) \leftarrow \textsc{Decode}(b,\ s_N)$
9: $\mathbf{w} \leftarrow \mathcal{W}[b_m B_M \,..\, (b_m{+}1)B_M)$ ▷ *worklist indirection*
10: $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
11: **for** $j = 0, \ldots, K_{\text{iters}}{-}1$ **do**
12: $(i, c_k) \leftarrow \textsc{Offset}(j)$ ▷ *kernel offset, channel tile*
13: $\mathbf{X} \leftarrow \mathbf{x}[\mathcal{N}_i(\mathbf{w}),\ c_k B_K{:}(c_k{+}1)B_K)$
14: $\mathbf{F} \leftarrow \mathbf{f}_i[c_k B_K{:}(c_k{+}1)B_K,\ b_n B_N{:}(b_n{+}1)B_N)$
15: $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{F}$
16: **end for**
17: $\mathbf{y}[\mathbf{w},\ b_n B_N{:}(b_n{+}1)B_N) \leftarrow \mathbf{Y}$ **where** $M_{out}[\mathbf{w}]{=}1$
18: **end for**
19: ▷ ——————————*Stream-K tail*——————————
20: **for** $b = \text{bid},\ \text{bid}+g_{\text{res}},\ \ldots < s_{\text{tail}} \times S_k$ **do**
21: $(b_m, b_n, r) \leftarrow \textsc{TailDecode}(b,\ s_N,\ S_k,\ s_{\text{dp}})$
22: $(k_{\text{lo}}, k_{\text{hi}}) \leftarrow \textsc{Partition}(K_{\text{iters}},\ S_k,\ r)$ ▷ *r-th K-slice*
23: $\mathbf{w} \leftarrow \mathcal{W}[b_m B_M \,..\, (b_m{+}1)B_M)$
24: $\mathbf{Y} \leftarrow \mathbf{0}_{B_M \times B_N}$
25: **for** $j = k_{\text{lo}}, \ldots, k_{\text{hi}}{-}1$ **do**
26: $(i, c_k) \leftarrow \textsc{Offset}(j)$
27: $\mathbf{X} \leftarrow \mathbf{x}[\mathcal{N}_i(\mathbf{w}),\ c_k B_K{:}(c_k{+}1)B_K)$
28: $\mathbf{F} \leftarrow \mathbf{f}_i[c_k B_K{:}(c_k{+}1)B_K,\ b_n B_N{:}(b_n{+}1)B_N)$
29: $\mathbf{Y} \leftarrow \mathbf{Y} + \mathbf{X} \cdot \mathbf{F}$
30: **end for**
31: $\mathbf{Z}[b_m, b_n][r] \leftarrow \mathbf{Y}$ ▷ *partial to workspace*
32: **if** $\textsc{LastArriver}(b_m, b_n)$ **then**
33: $\mathbf{Y} \leftarrow \sum_{r'=0}^{S_k-1} \mathbf{Z}[b_m, b_n][r']$
34: $\mathbf{y}[\mathbf{w},\ b_n B_N{:}(b_n{+}1)B_N) \leftarrow \mathbf{Y}$ **where** $M_{out}[\mathbf{w}]{=}1$
35: **end if**
36: **end for**

6 Implementation

We implement WISEConv as a PyTorch extension with a Python frontend and a CUDA backend written in C++. A model-conversion pass replaces supported modules, preserves their parameters and weights, and statically marks worklist-reuse edges and rebuild points. The compute backend adapts CUTLASS [1] dense tiled implicit-GEMM pipelines for direct worklist consumption, covering both FP32 (SIMT) and FP16 (tensor core) arithmetic with NHWC tensors; the full-workload comparison uses the FP32 datapath, while sensitivity studies additionally exercise the FP16 datapath. The backend covers all convolution variants used by the evaluated models, as well as mask-aware auxiliary operators. The partial-reduction workspace $\mathbf{Z}$ is shared across all layers at the maximum required size and allocated once during initialization. Every backend variant agrees numerically with cuDNN at each active output position.

7 Evaluation

7.1 Experimental Setup

Platforms. We evaluate six GPU platforms. The four discrete GPUs are NVIDIA A100, RTX 4090, RTX 3080, and RTX 4070 Laptop. The embedded platforms are Jetson Xavier NX and AGX Orin; we also measure board energy on both. The discrete platforms use PyTorch 2.1.2, CUDA 12.1, and cuDNN 8.9.2; both Jetson platforms use PyTorch 2.1.0, CUDA 11.4, and cuDNN 8.6. All paths in the full workload comparison use FP32 (SIMT) with TF32 disabled. Sensitivity studies additionally evaluate FP16 (tensor core) on selected platforms. On each platform, all backends run under the same fixed clock and power configuration.

Models, tasks, and mask sources. Our workloads (Table 2) span three vision tasks: FireFlowNet [29] for event-based optical flow on MVSEC [46] with Ev-Conv [9] masks, YOLOv8n and YOLOv8m [20] for detection on MOT16 [26] with DeltaCNN's [31] temporal masks, and DynConv [38] for human pose on MPII [2] with learned spatial gates. For every workload, calibration uses the training split, while all reported results use a disjoint test split with the authors' released weights [20, 29, 38]. Within each workload, we fix inputs, weights, and upstream mask policy; every sparse path receives the same layer masks, isolating the masked-convolution backend.

Baselines. We compare WISEConv against three FP32 (SIMT) execution paths: **Dense** (PyTorch/cuDNN [7] with autotuning), **Tile skipping** (DeltaCNN [31] for FireFlowNet and YOLO; DynConv supplies output masks directly), and **Gather-scatter** (DynConv's [38] CUDA backend for pose; MMCV's [6] MaskedConv2d for FireFlowNet and YOLO).

Measurements. We measure per-frame full-model GPU latency over the complete evaluation datasets. CUDA events

Table 2. Evaluation workloads.

	FireFlowNet [29]	**YOLOv8n/m** [20]	**DynConv** [38]
Task	Optical flow	Detection	Human pose
Mask source	Ev-Conv [9]	DeltaCNN [31]	DynConv [38]
Dataset	MVSEC [46]	MOT16 [26]	MPII [2]
Input size	256×256	640×640	256×256
Parameters	0.056M	3.16M / 25.9M	6.89M
Eval. samples	196.8K	2,572	2,958

bracket each frame's inference; after warm-up we run three complete rounds and average per-frame latency across all frames and rounds. The interval includes mask propagation, worklist construction, all network operators, and output update; it excludes input transfer, offline calibration, and one-time initialization. We report η, T, and T_{eff} at the model level by accumulating required and issued work across layers per frame (Eq. 3), then averaging across frames. On the Jetson platforms, we integrate board-level `VDD_IN` power over each frame interval to obtain per-frame energy. For accuracy, we follow the original protocols: AEE for optical flow [9], AP for detection [31], and PCKh for pose [38]. MOT16 AP uses fixed pseudo-labels from a dense YOLO26x teacher [21] because ground-truth detections cover only the training split.

7.2 End-to-End Performance

Latency. WISEConv is the fastest path in all 24 measured workload-platform pairs (Figure 8). Relative to the fastest evaluated competing path in each pair, it achieves a 1.58× geometric-mean speedup; individual-pair speedups range from 1.33× to 2.12×, corresponding to 25.0–52.8% lower latency. Gains span more than two orders of magnitude in absolute latency, from 0.23 ms for FireFlowNet on RTX 4090 to 88.6 ms for YOLOv8m on Xavier NX.

The two existing sparse paths split the remaining 24 pairs between them: dense is fastest in 13 and tile skipping in 11; gather-scatter never wins. The split tracks active ratio: at the YOLO workloads' $\alpha \approx 48$–49%, few tiles are fully inactive, so tile skipping cannot compensate for its lower throughput; on FireFlowNet ($\alpha \approx 25\%$) and DynConv ($\alpha \approx 16\%$), enough tiles are skippable for it to overtake dense. Neither path is uniformly strong, and WISEConv outperforms whichever wins. On the Jetson platforms the lead narrows (geometric mean 1.53× vs. 1.61× on discrete GPUs) but WISEConv remains fastest in all eight pairs.

Across all 24 pairs, WISEConv averages 1.76× the ideal proportional-scaling reference (dashed lines in Figure 8), compared with 2.79× for the per-pair fastest competing path. The gap to ideal tracks active ratio: on the YOLO workloads WISEConv runs at 1.36× ideal, while on DynConv ($\alpha \approx 16\%$) construction and fixed costs widen it to 2.60×.

Energy. WISEConv is the lowest-energy path for every workload on both Jetson platforms (Table 3), reducing energy by 9.3–36.1% relative to the lowest-energy competing path in each pair and by 29.9–67.8% relative to dense execution.

Accuracy. WISEConv's latency and energy gains do not alter accuracy. WISEConv matches the other sparse paths on every workload: AEE and PCKh are identical and YOLOv8 AP differs by at most 0.14 points, a residual of floating-point reduction order rather than any difference in covered positions (Figure 9); even where worklist reuse computes a superset of the required positions, the output mask gates writeback so the extra slots do not reach downstream layers. The remaining accuracy gap between sparse and dense, within 4% across all workloads, is set by the upstream mask policy, which WISEConv consumes without modification.

7.3 Microbenchmarks

Effective throughput. A sparse path lowers latency only if it is both selective, captured by the useful-compute ratio η, and fast, captured by raw throughput T (Eq. 4). Figure 10 reports both, normalized to Dense, on RTX 3080 and AGX Orin, and the two existing paths fail on opposite axes. Tile skipping retains the dense tile schedule and so keeps 41–65% of Dense raw throughput, but an active tile computes every position it covers: its useful-compute ratio falls to 0.45–0.72, wasting 28–55% of the slots it issues. Gather-scatter issues exactly the required positions, giving $\eta = 1$, but extraction and writeback collapse its throughput to 2–47% of Dense. WISEConv is the only path that is high on both axes, sustaining $\eta = 0.90$–1.00 and the highest raw throughput among the sparse paths in every workload—42.0–85.1% of Dense on RTX 3080 and 51.3–86.8% on AGX Orin. The two YOLO workloads lie at the upper end because their higher active ratios expose more parallelism and amortize fixed costs; DeltaCNN's throughput on FireFlowNet and DynConv reflects routing very sparse tiles through its fused selective path [31]. Because effective throughput is the product of the two axes, WISEConv reaches 1.38–2.06× the strongest competing effective throughput on RTX 3080 and 1.47–1.87× on AGX Orin.

Table 3. Per-frame board energy (J) on the Jetson platforms. Parentheses report each sparse path's change relative to Dense.

GPU	System	FireFlowNet	YOLOv8n	YOLOv8m	DynConv
AGX Orin	Dense	0.19	0.34	1.82	1.01
	Tile skip	0.10 (-47.9%)	0.31 (-8.0%)	1.59 (-12.7%)	0.51 (-49.7%)
	Gather	0.27 (+44.8%)	0.62 (+83.9%)	1.88 (+3.3%)	2.49 (+145.5%)
	WISEConv	**0.08** (-58.3%)	**0.22** (-35.6%)	**1.06** (-42.0%)	**0.33** (-67.8%)
Xavier NX	Dense	0.28	0.49	2.57	1.17
	Tile skip	0.14 (-48.7%)	0.41 (-14.9%)	2.31 (-10.2%)	0.60 (-48.7%)
	Gather	0.38 (+36.4%)	0.79 (+63.0%)	2.90 (+12.7%)	3.08 (+163.3%)
	WISEConv	**0.13** (-53.5%)	**0.34** (-29.9%)	**1.78** (-30.8%)	**0.51** (-56.8%)

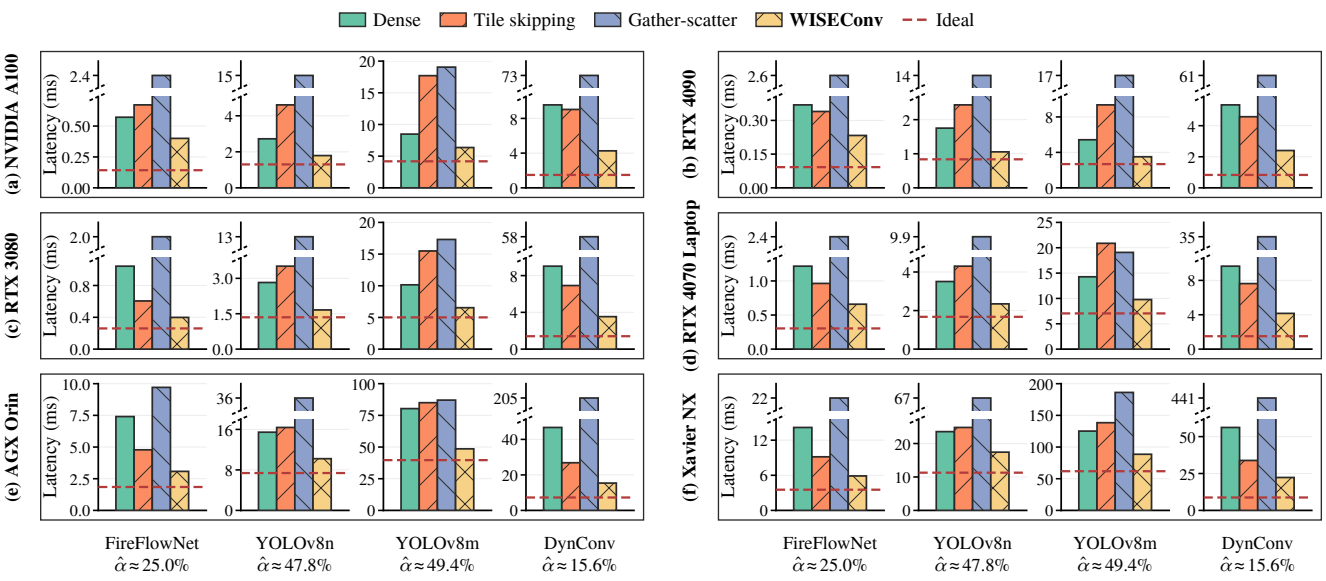


Figure 8. Full-model mean per-frame latency across GPU platforms. Each workload uses an independent y-axis. Labels report the model-level active ratio $\hat{\alpha}$, aggregated as described in §7.1 and averaged across evaluation sequences. Red dashed lines mark ideal proportional scaling, computed by applying each sequence's model-level active ratio to its dense latency before aggregation.

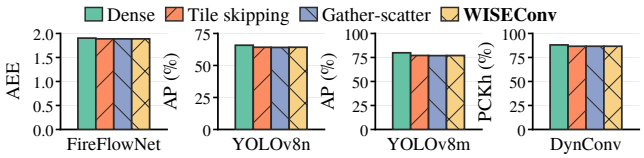


Figure 9. Task accuracy on RTX 3080 under the evaluated upstream sparsity policies, measured by AEE (↓), AP (↑), and PCKh (↑).

Table 4. WISEConv η decomposition and worklist-length statistics on RTX 3080 (FP32; batch 1 unless noted). The $n_{\mathcal{W}}$ percentiles are computed from worklist lengths pooled across all layers and frames.

Workload	$\eta_{\text{construct}}$	η_{compute}	η	$n_{\mathcal{W}}$		
				P10	P50	P90
FireFlowNet	100.0%	99.8%	99.8%	529	13,651	36,009
YOLOv8m	100.0%	92.4%	92.4%	24	261	1,481
YOLOv8m b4	100.0%	98.1%	98.1%	184	1,410	7,256
DynConv	100.0%	92.1%	92.1%	3	43	389

Table 4 localizes WISEConv's own loss on RTX 3080 through the decomposition of Eq. 5. $\eta_{\text{construct}}$, the fraction of worklist entries that are required positions, stays at 98.6–100.0%: tile-local compaction emits each active coordinate exactly once, and only reuse of a superset worklist gives up the remainder. η_{compute}, the fraction of issued slots that are worklist entries, exceeds 99.5% for FireFlowNet and the two YOLO workloads. DynConv is the exception at 90.6%: its low active ratio and small spatial grids yield exceptionally short worklists (P10, P50, and P90 lengths of 3, 43, and 389), so B_M-sized final tiles are mostly padding. Because its inputs are temporally unrelated, DynConv uses the same exact-length device policy without a temporal activity selector. Even at this worst case the total η is more than double tile skipping's 44.8% on the same workload.

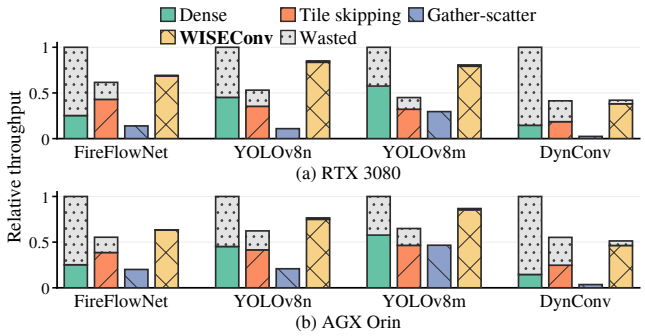


Figure 10. Full-model effective throughput ηT and wasted throughput $(1-\eta)T$ on RTX 3080 and AGX Orin, normalized within each platform to Dense raw throughput.

Latency cost decomposition. We decompose full-model YOLOv8n latency by execution stage on RTX 3080 and AGX

Table 5. Full-model YOLOv8m latency decomposition (batch 1, RTX 3080).

	System	Tot.	Cons.	Conv.	G/S	Other	Idle
FP32	Dense	10.76	—	9.68 (90)	—	1.08 (10)	—
	Tile skipping	12.51	—	11.70 (94)	—	0.81 (6)	—
	Gather-scatter	12.74	0.37 (3)	1.68 (13)	1.58 (12)	0.84 (7)	8.26 (65)
	WISEConv	3.00	0.17 (6)	2.27 (76)	—	0.56 (19)	—
FP16	Dense	3.80	—	2.69 (71)	—	1.11 (29)	—
	WISEConv	1.44	0.15 (10)	0.88 (61)	—	0.42 (29)	—

Notes. Entries are milliseconds per frame; parenthesized numbers are percentages of each row's total, the whole-video end-to-end latency. Stages are the kernels' shares of the GPU busy time (nsys); Cons. is the worklist construction (gather-scatter's mask compaction), G/S gather-scatter's feature extraction and scatter, Other the remaining operators; Idle is the GPU idle time within the frame (host synchronization). The baselines have no FP16 implementation.

Orin (Table 5). On RTX 3080, WISEConv reduces the convolution share from 2.32 to 0.99 ms (2.3×, close to the $1/\alpha \approx 2.2$ proportional target) while adding 0.12 ms of construction, lowering the total from 2.82 to 1.66 ms. On AGX Orin the convolution reduction is 2.0× (11.91 to 6.06 ms) at 0.51 ms of construction, and the total falls from 15.45 to 10.22 ms.

Gather-scatter is the exception: it issues exactly the required positions, so its convolution is the smallest share of any path (30% of its total), but extraction, scatter, and buffer setup cost 7.48 ms, 59% of its total, on RTX 3080. Exact selection alone does not lower latency without a fused datapath.

Memory overhead. Table 6 reports peak live GPU allocation. FireFlowNet, YOLOv8n, and YOLOv8m use temporal mask policies that retain previous-frame activations for differencing, so every sparse path allocates 3–4× Dense. The temporal state dominates the gap, and WISEConv stays within the same tier as tile skipping; gather-scatter is the exception on YOLOv8m, where materializing input patches for each active position adds to the temporal state and pushes it above both other sparse paths. DynConv applies learned spatial gates without temporal state, placing all backends in a narrow 27–34 MiB range. Here WISEConv's worklist and coordinate metadata add only 2.3 MiB over Dense, while gather-scatter is the most compact because DynConv's predominantly pointwise block structure lets intermediates stay in the gathered compact representation [38].

Table 6. Peak live GPU allocation (MiB) on RTX 3080 (FP32; batch 1 unless noted).

System	FireFlowNet	YOLOv8m	YOLOv8m b4	DynConv
Dense	33.9	182.0	432.2	56.8
Tile skipping	141.8	751.0	2701.0	53.8
Gather-scatter	197.0	834.8	2737.0	57.2
WISEConv	128.6	763.9	2730.4	56.9

Table 7. Co-design ablation on YOLOv8m (batch 1, RTX 3080, FP16). Each variant replaces one mechanism and keeps the weights, inputs, upstream masks and every other component. Cons. and Conv. are the construction- and compute-stage GPU kernel time (ms); Tot. is the full-model latency (ms, aligned to the whole-video end-to-end latency of Ours); × is relative to Ours unless noted.

	Variant	Cons.	Conv.	Tot.	×
Cons.	No reuse	0.30	0.86	1.59	1.10
Comp.	Shuffled order	0.15	0.92	—	1.04†
	Readback & launch	—	—	9.40	6.51
	Full-layer grid	0.15	1.06	1.63	1.13†
Sel.	Dense-tuned tile	0.16	1.89	2.47	1.71
	Best single tile	0.16	0.88	1.47	1.02
	Latency-optimal tile	0.16	1.00	1.59	1.10
	Cost-unaware $K_{\max}$	0.22	0.85	1.55	1.07
	Zero-cost oracle	—	—	1.32	0.92
	Ours	0.16	0.86	1.44	1.00

Variants. No reuse: rebuild the worklist at every layer. *Shuffled order*: permute each worklist on the device before the compute stage (permutation not counted; convolution time). *Readback & launch*: read n_W back to the host per layer and launch the exact grid. *Full-layer grid*: launch the dense layer's grid; CTAs beyond the worklist exit. *Dense-tuned / Best single / Latency-optimal tile*: Ours' candidate structure (the same layers, candidate counts and thresholds), every candidate the tile fastest at α=1, with the best expected speedup, or with the lowest expected latency (uniform n_W). *Cost-unaware* $K_{\max}$: up to four candidates at every layer, ignoring their cost. *Zero-cost oracle*: those candidates, each frame on a graph captured for its own n_W, with no selection mechanism. †Relative to the best single tile without the selection mechanism.

7.4 Ablation Study and Sensitivity Analysis

Table 7 isolates each co-designed component by disabling it while holding all others fixed on YOLOv8n; the stage decomposition separates the effect on construction and convolution.

Construction. On RTX 3080, Atomic append and Global order raise construction from 0.12 to 0.43 and 0.54 ms without reducing convolution (0.94 ms throughout), adding 19.1% and 19.6% to the end-to-end latency over Ours; Global order's higher construction reflects its additional kernel passes. No reuse *raises* η to 99.7% ($\eta_{\text{construct}} = 1$ per layer) but increases construction 1.6× (0.12 to 0.19 ms), adding 5.7% over Ours: rebuilding the worklist every layer is more expensive than reusing a slightly loose superset worklist. On AGX Orin the same pattern holds with smaller margins: construction variants add 1–9% over Ours while convolution stays within 0.05 ms, because convolution is a larger share of the total.

Compute. On RTX 3080, DP only raises convolution from 0.94 to 1.01 ms over Ours (+0.07 ms); No hybrid, which retains full Stream-K but not the hybrid mode, recovers most of that gap (0.96 ms, +0.02 ms over Ours). Readback & launch runs the same kernels as Ours but adds per-operator host synchronization, inflating the total from 1.66 to 7.61 ms (4.58×

Ours). On AGX Orin, DP only adds 1.5% end-to-end latency over Ours and readback inflates it to 2.38× Ours.

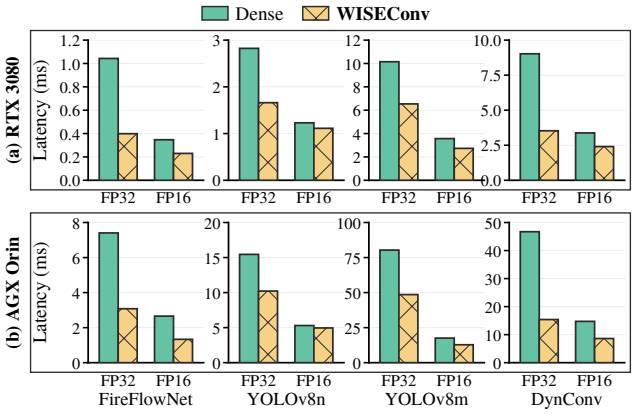


Figure 11. FP32 (SIMT) vs. FP16 (tensor core) full-model latency for Dense and WISEConv across all four workloads on RTX 3080 and AGX Orin.

Tensor core sensitivity. Figure 11 compares FP32 and FP16 execution on RTX 3080 and AGX Orin. The evaluated sparse baselines [6, 31, 38] provide only FP32 paths, so this study compares Dense and WISEConv only. WISEConv retains a 1.07–1.99× speedup over Dense under FP16 tensor cores across all eight workload-platform pairs, narrowing from 1.51–3.04× under FP32. The narrowing is expected: tensor cores reduce the convolution time that WISEConv saves by skipping, shrinking the absolute saving while non-convolution costs (construction, elementwise, mask processing) do not shrink proportionally. The two YOLO workloads narrow the most (1.51–1.70× under FP32 to 1.07–1.37× under FP16) because their high active ratio ($\alpha \approx 48$–49%) already limits the sparsity gain; within the two, YOLOv8n narrows further because its smaller model makes non-convolution costs a larger share of the total under FP16.

Stream-level batching. Table 8 reports per-frame latency decomposition for YOLOv8n on RTX 3080 at batch sizes 1, 4, and 8. WISEConv remains fastest at every batch size, but its lead over the fastest competing path narrows from 1.70× at batch 1 to 1.59× at batch 8. Tile skipping benefits most: from batch 1 to 8, its per-frame convolution drops from 2.89 to 0.92 ms, compared with 0.99 to 0.49 ms for WISEConv, because batching adds independent tiles that fill otherwise idle SMs [31], the same occupancy that WISEConv's adaptive decomposition already provides at batch 1. Tile skipping accordingly overtakes dense at batch 4 and stays the fastest competing path at batch 8. Gather-scatter's extraction overhead does not amortize: *Other* still accounts for 59.4% of its per-frame latency at batch 8.

Table 8. YOLOv8m latency decomposition under stream-level batching (RTX 3080, FP32). Each batch combines one current frame of each of B disjoint temporal streams; times are per frame. Stage definitions and cell format follow Table 5.

	System	Tot.	Cons.	Conv.	G/S	Other	Idle
B=1	Dense	10.76	—	9.68 (90)	—	1.08 (10)	—
	Tile skipping	12.51	—	11.70 (94)	—	0.81 (6)	—
	Gather-scatter	12.74	0.37 (3)	1.68 (13)	1.58 (12)	0.84 (7)	8.26 (65)
	WISEConv	3.00	0.17 (6)	2.27 (76)	—	0.56 (19)	—
B=4	Dense	11.04	—	9.90 (90)	—	1.12 (10)	0.02 (0)
	Tile skipping	5.04	—	4.46 (88)	—	0.59 (12)	—
	Gather-scatter	4.50	0.13 (3)	1.42 (32)	1.50 (33)	0.56 (12)	0.89 (20)
	WISEConv	1.88	0.05 (3)	1.49 (79)	—	0.33 (18)	0.01 (0)

7.5 Discussion

WISEConv consumes rather than generates masks, so its gains are conditional on the upstream policy and its activity distribution; it guarantees coverage of requested outputs but does not improve mask quality. When the active ratio is high enough, construction overhead and the gap between the worklist kernel and cuDNN leave little room for a net gain, as the YOLO workloads' narrower speedups already suggest.

The adaptive decomposition is calibrated per convolution shape and device; results are cached and reused across models that share the same shapes, but a new shape or device requires reprofiling. Because calibration selects only among decomposition modes, it affects performance but not coverage or correctness. The current calibration policy is simple; a faster or more effective one is a natural direction for future work.

The full workload comparison uses FP32 (SIMT) because the evaluated sparse baselines [6, 31, 38] provide only FP32 paths. The sensitivity study confirms that WISEConv retains end-to-end gains under FP16 tensor cores on both evaluated platforms across all four workloads. Extending the worklist compute stage to native tensor-core reduction remains open. Stream-K performs FP32 partial-sum reduction through an external workspace (§5.3), which is necessary for numerical precision but adds overhead that, for the short worklists where split-K is most needed, can outweigh the tensor-core arithmetic savings. A broader study across GPU architectures remains for future work.

8 Conclusion

We presented WISEConv, a masked-convolution runtime that replaces the dense tiled pipeline's contiguous coordinate stream with an active-output worklist, computing almost exclusively the positions an upstream mask requires while retaining the dense tensor layout and the regular per-position reduction. The construction and consumption stages

are co-designed to secure both useful-compute ratio and throughput; our ablation confirms that disabling any single mechanism degrades one axis without improving the other. The resulting higher effective throughput lets WISEConv convert upstream mask sparsity into proportional latency and energy reduction for the latency-critical vision tasks where masked convolution applies.

References

[1] 2026. CUTLASS: CUDA Templates and Python DSLs for High-Performance Linear Algebra. https://github.com/NVIDIA/cutlass

[2] Mykhaylo Andriluka, Leonid Pishchulin, Peter Gehler, and Bernt Schiele. 2014. 2D Human Pose Estimation: New Benchmark and State of the Art Analysis. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3686–3693.

[3] Rik J. Bouwmeester, Federico Paredes-Vallés, and Guido C. H. E. de Croon. 2023. NanoFlowNet: Real-time Dense Optical Flow on a Nano Quadcopter. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 1996–2003.

[4] Lukas Cavigelli, Philippe Degen, and Luca Benini. 2017. CBinfer: Change-Based Inference for Convolutional Neural Networks on Video Data. In *Proc. Int. Conf. Distrib. Smart Cameras (ICDSC)*. 1–8.

[5] Tatjana Chavdarova, Pierre Baqué, Stéphane Bouquet, Andrii Maksai, Cijo Jose, Timur M. Bagautdinov, Louis Lettry, Pascal Fua, Luc Van Gool, and François Fleuret. 2018. WILDTRACK: A Multi-Camera HD Dataset for Dense Unscripted Pedestrian Detection. In *2018 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2018, Salt Lake City, UT, USA, June 18-22, 2018*. Computer Vision Foundation / IEEE Computer Society, 5030–5039. doi:10.1109/CVPR.2018.00528

[6] Kai Chen, Jiaqi Wang, Jiangmiao Pang, Yuhang Cao, Yu Xiong, Xiaoxiao Li, Shuyang Sun, Wansen Feng, Ziwei Liu, Jiarui Xu, Zheng Zhang, Dazhi Cheng, Chenchen Zhu, Tianheng Cheng, Qijie Zhao, Buyu Li, Xin Lu, Rui Zhu, Yue Wu, Jifeng Dai, Jingdong Wang, Jianping Shi, Wanli Ouyang, Chen Change Loy, and Dahua Lin. 2019. MMDetection: Open MMLab Detection Toolbox and Benchmark. *CoRR* abs/1906.07155 (2019).

[7] Sharan Chetlur, Cliff Woolley, Philippe Vandermersch, Jonathan Cohen, John Tran, Bryan Catanzaro, and Evan Shelhamer. 2014. cuDNN: Efficient Primitives for Deep Learning. *CoRR* abs/1410.0759 (2014).

[8] Christopher B. Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.

[9] Sankeerth Durvasula, Yushi Guan, and Nandita Vijaykumar. 2023. EvConv: Fast CNN Inference on Event Camera Inputs for High-Speed Robot Perception. *IEEE Robotics Autom. Lett.* 8, 6 (2023), 3174–3181.

[10] Davide Falanga, Philipp Foehn, Peng Lu, and Davide Scaramuzza. 2018. PAMPC: Perception-Aware Model Predictive Control for Quadrotors. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018, Madrid, Spain, October 1-5, 2018*. IEEE, 1–8. doi:10.1109/IROS.2018.8593739

[11] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How Fast Is Too Fast? The Role of Perception Latency in High-Speed Sense and Avoid. *IEEE Robotics Autom. Lett.* 4, 2 (2019), 1884–1891. doi:10.1109/LRA.2019.2898117

[12] Davide Falanga, Kevin Kleber, and Davide Scaramuzza. 2020. Dynamic obstacle avoidance for quadrotors with event cameras. *Sci. Robotics* 5, 40 (2020), 9712. doi:10.1126/SCIROBOTICS.AAZ9712

[13] Ruibo Fan, Wei Wang, and Xiaowen Chu. 2024. DTC-SpMM: Bridging the Gap in Accelerating General Sparse Matrix Multiplication with Tensor Cores. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 253–267. doi:10.1145/3620666.3651378

[14] Michael Figurnov, Maxwell D. Collins, Yukun Zhu, Li Zhang, Jonathan Huang, Dmitry P. Vetrov, and Ruslan Salakhutdinov. 2017. Spatially Adaptive Computation Time for Residual Networks. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 1790–1799.

[15] Trevor Gale, Matei Zaharia, Cliff Young, and Erich Elsen. 2020. Sparse GPU Kernels for Deep Learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

[16] Yizhao Gao, Baoheng Zhang, Xiaojuan Qi, and Hayden Kwok-Hay So. 2023. DPACS: Hardware Accelerated Dynamic Neural Network Pruning through Algorithm-Architecture Co-design. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2023, Vancouver, BC, Canada, March 25-29, 2023*, Tor M. Aamodt, Natalie D. Enright Jerger, and Michael M. Swift (Eds.). ACM, 237–251. doi:10.1145/3575693.3575728

[17] Benjamin Graham, Martin Engelcke, and Laurens van der Maaten. 2018. 3D Semantic Segmentation with Submanifold Sparse Convolutional Networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

[18] Amirhossein Habibian, Davide Abati, Taco S. Cohen, and Babak Ehteshami Bejnordi. 2021. Skip-Convolutions for Efficient Video Processing. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2695–2704.

[19] Drew Hanover, Antonio Loquercio, Leonard Bauersfeld, Angel Romero, Robert Penicka, Yunlong Song, Giovanni Cioffi, Elia Kaufmann, and Davide Scaramuzza. 2024. Autonomous Drone Racing: A Survey. *IEEE Trans. Robotics* 40 (2024), 3044–3067. doi:10.1109/TRO.2024.3400838

[20] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. 2023. Ultralytics YOLOv8. https://github.com/ultralytics/ultralytics

[21] Glenn Jocher, Jing Qiu, Mengyu Liu, Shuai Lyu, Fatih Cagatay Akyon, and Muhammet Esat Kalfaoglu. 2026. Ultralytics YOLO26: Unified Real-Time End-to-End Vision Models. *CoRR* abs/2606.03748 (2026).

[22] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman P. Amarasinghe. 2017. The Tensor Algebra Compiler. *Proc. ACM Program. Lang.* 1, OOPSLA (2017).

[23] Fanrong Li, Gang Li, Xiangyu He, and Jian Cheng. 2021. Dynamic Dual Gating Neural Networks. In *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*. 5310–5319.

[24] Renyuan Liu, Yuyang Leng, Shilei Tian, Shaohan Hu, Chun-Fu (Richard) Chen, and Shuochao Yao. 2024. DynaSpa: Exploiting Spatial Sparsity for Efficient Dynamic DNN Inference on Devices. In *Proceedings of the 22nd ACM Conference on Embedded Networked Sensor Systems, SenSys 2024, Hangzhou, China, November 4-7, 2024*. ACM, 422–435. doi:10.1145/3666025.3699348

[25] Nico Messikommer, Daniel Gehrig, Antonio Loquercio, and Davide Scaramuzza. 2020. Event-Based Asynchronous Sparse Convolutional Networks. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 415–431.

[26] Anton Milan, Laura Leal-Taixé, Ian D. Reid, Stefan Roth, and Konrad Schindler. 2016. MOT16: A Benchmark for Multi-Object Tracking. *CoRR* abs/1603.00831 (2016).

[27] Asit K. Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. 2021. Accelerating Sparse Deep Neural Networks. *CoRR* abs/2104.08378 (2021).

[28] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D. Owens. 2023. Stream-K: Work-centric Parallel Decomposition for Dense Matrix-Matrix Multiplication on the GPU. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. doi:10.1145/3572848.3577479

[29] Federico Paredes-Vallés and Guido C. H. E. de Croon. 2021. Back to Event Basics: Self-Supervised Learning of Image Reconstruction for Event Cameras via Photometric Constancy. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3445–3454.

[30] Mathias Parger, Chengcheng Tang, Thomas Neff, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2023. MotionDeltaCNN: Sparse CNN Inference of Frame Differences in Moving Camera Videos with Spherical Buffers and Padded Convolutions. In *IEEE/CVF International Conference on Computer Vision, ICCV 2023, Paris, France, October 1-6, 2023*. 17246–17255.

[31] Mathias Parger, Chengcheng Tang, Christopher D. Twigg, Cem Keskin, Robert Wang, and Markus Steinberger. 2022. DeltaCNN: End-to-End CNN Inference of Sparse Frame Differences in Videos. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 12487–12496.

[32] Hongwu Peng, Xi Xie, Kaustubh Shivdikar, Md Amit Hasan, Jiahui Zhao, Shaoyi Huang, Omer Khan, David R. Kaeli, and Caiwen Ding. 2024. MaxK-GNN: Extremely Fast GPU Kernel Design for Accelerating Graph Neural Networks Training. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, ASPLOS 2024, La Jolla, CA, USA, 27 April 2024- 1 May 2024*, Rajiv Gupta, Nael B. Abu-Ghazaleh, Madan Musuvathi, and Dan Tsafrir (Eds.). ACM, 683–698. doi:10.1145/3620665.3640426

[33] Mengye Ren, Andrei Pokrovsky, Bin Yang, and Raquel Urtasun. 2018. SBNet: Sparse Blocks Network for Fast Inference. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 8711–8720.

[34] Spconv Contributors. 2022. Spconv: Spatially Sparse Convolution Library. https://github.com/traveller59/spconv.

[35] Martin Sundermeyer, Arsalan Mousavian, Rudolph Triebel, and Dieter Fox. 2021. Contact-GraspNet: Efficient 6-DoF Grasp Generation in Cluttered Scenes. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 13438–13444.

[36] Haotian Tang, Zhijian Liu, Xiuyu Li, Yujun Lin, and Song Han. 2022. TorchSparse: Efficient Point Cloud Inference Engine. In *Proceedings of Machine Learning and Systems (MLSys)*.

[37] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *Proc. IEEE/ACM Int. Symp. Microarchitecture (MICRO)*. 225–239.

[38] Thomas Verelst and Tinne Tuytelaars. 2020. Dynamic Convolutions: Exploiting Spatial Sparsity for Faster Inference. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 2320–2329.

[39] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martin Martin, Cewu Lu, Li Fei-Fei, and Silvio Savarese. 2019. DenseFusion: 6D Object Pose Estimation by Iterative Dense Fusion. In *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*. 3343–3352.

[40] Kunyun Wang, Shuo Yang, Jieru Zhao, Wenchao Ding, Quan Chen, Jingwen Leng, and Minyi Guo. 2025. SparseTem: Boosting the Efficiency of CNN-Based Video Encoders by Exploiting Temporal Continuity. In *Advanced Parallel Processing Technologies - 16th International Symposium, APPT 2025, Athens, Greece, July 13-16, 2025, Proceedings*. 246–256.

[41] Diana Wofk, Fangchang Ma, Tien-Ju Yang, Sertac Karaman, and Vivienne Sze. 2019. FastDepth: Fast Monocular Depth Estimation on Embedded Systems. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*. 6101–6108.

[42] Zhenda Xie, Zheng Zhang, Xizhou Zhu, Gao Huang, and Stephen Lin. 2020. Spatially Adaptive Inference with Stochastic Feature Sampling and Interpolation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 531–548.

[43] Jiacheng Yang, Christina Giannoula, Jun Wu, Mostafa Elhoushi, James Gleeson, and Gennady Pekhimenko. 2024. Minuet: Accelerating 3D Sparse Convolutions on GPUs. In *Proceedings of the Nineteenth European Conference on Computer Systems, EuroSys 2024, Athens, Greece, April 22-25, 2024*. 786–802.

[44] Zihao Ye, Ruihang Lai, Junru Shao, Tianqi Chen, and Luis Ceze. 2023. SparseTIR: Composable Abstractions for Sparse Compilation in Deep Learning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*.

[45] Changqian Yu, Jingbo Wang, Chao Peng, Changxin Gao, Gang Yu, and Nong Sang. 2018. BiSeNet: Bilateral Segmentation Network for Real-Time Semantic Segmentation. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*. 334–349.

[46] Alex Zihao Zhu, Dinesh Thakur, Tolga Özaslan, Bernd Pfrommer, Vijay Kumar, and Kostas Daniilidis. 2018. The Multi Vehicle Stereo Event Camera Dataset: An Event Camera Dataset for 3D Perception. *CoRR* abs/1801.10202 (2018).